preface

I first encountered cross-origin requests around 2006, when I joined Google and became the owner of the GData JavaScript Client. The GData JavaScript Client was a library that gave developers access to various Google APIs from JavaScript. The library itself was written in JavaScript, and the code was pretty straightforward...except for this little corner of code that made cross-origin requests to Google's servers. This was before CORS existed, so this little corner jumped though crazy hoops to load data from Google's APIs. From the developer's perspective, the code simply worked. But between the request and the response was a dark and convoluted maze of code that was difficult to understand and debug.

So you can imagine my happiness when I discovered CORS. Here was a clean, simple, and standard way for making cross-origin requests. Instead of code that's difficult to understand, I could have simple HTTP response headers. Instead of code that's difficult to debug, I could have a single standard that worked across all browsers. I quickly set out to add support for CORS to Google's APIs.

And that's when the real fun started. While CORS uses HTTP headers to enable cross-origin requests, there are many subtle ways in which these headers can interact. It's not as simple as adding an HTTP header to your server and calling it a day. And because CORS was such a new feature, there weren't a lot of resources to guide me. Armed with the CORS spec, Wireshark, and a lot of patience, I spent the next few weeks building a flexible and configurable CORS library that could work for various types of requests. Based on that experience, I started contributing CORS knowledge to